

```

1 (* Alberi binari *)
2
3 type expr =
4   Int of int
5 | Var of string
6 | Sum of expr * expr
7 | Diff of expr * expr
8 | Mult of expr * expr
9 | Div of expr * expr
10
11 let rec subexpr e1 e2 = match e1 with
12   | Int _ | Var _ -> e1 = e2
13   | Sum(e',e'') | Diff(e',e'') | Mult(e',e'') | Div(e',e'') ->
14     e1=e2 || subexpr e' e2 || subexpr e'' e2
15
16 let rec subst_in_expr e x e' = match e with
17   | Int _ -> e
18   | Var y -> if x=y then e' else e
19   | Sum(e1,e2) -> Sum (subst_in_expr e1 x e', subst_in_expr e2 x e')
20   | Diff(e1,e2) -> Diff (subst_in_expr e1 x e', subst_in_expr e2 x e')
21   | Mult(e1,e2) -> Mult (subst_in_expr e1 x e', subst_in_expr e2 x e')
22   | Div(e1,e2) -> Div (subst_in_expr e1 x e', subst_in_expr e2 x e')
23 ;;
24
25 (* Visite alberi binari *)
26 type 'a tree = Empty | Tr of 'a * 'a tree * 'a tree;;
27
28 let rec reflect = function
29   | Empty -> Empty
30   | Tr(x,tr1,tr2) -> Tr(x, reflect tr2, reflect tr1);;
31
32 let rec take n = function
33   | [] -> []
34   | x::xs -> if n<=0 then []
35               else x::(take (n-1) xs)
36
37 let rec drop n = function
38   | [] -> []
39   | l -> if n>0 then drop (n-1) (List.tl l)
40           else l
41
42
43 let rec balpreorder list =
44   match list with
45   | [] -> Empty
46   | x::rest -> let lunghezza = List.length list in
47     Tr(x, balpreorder(take(lunghezza/2) rest),
48 balpreorder(drop(lunghezza/2) rest));;
49
50 let rec balinorder list =
51   match list with
52   | [] -> Empty
53   | lst -> let centro = (List.length lst)/2 in
54     let elementoCentrale = List.nth lst centro in
55     let listaSinistra = take centro lst in
56     let listaDestra = drop (centro+1) lst in
57     Tr(elementoCentrale, balinorder(listaSinistra),
58 balinorder(listaDestra));;
59
60 let balpostorder l = reflect(balpreorder (List.rev l))

```

```

59 ;;
60
61 let subset l1 l2 = List.for_all (function x -> List.mem x l2) l1;;
62
63
64 let rec foglie_in_lista list albero =
65     match albero with
66     | Empty -> true
67     | Tr(x, Empty, Empty) -> if (List.mem x list) then true else false
68     | Tr(x, l, r) -> foglie_in_lista list l && foglie_in_lista list r;;
69
70 exception AlberoVuoto
71
72 let rec segui_bool list albero=
73     match albero with
74     | Empty -> raise AlberoVuoto
75     | Tr(x, l, r) -> match list with
76     | [] -> x
77     | y::rest -> if y then segui_bool rest l else segui_bool rest
78     r;;
79
80 let rec altezza albero =
81     match albero with
82     | Empty -> 0
83     | Tr(_, l, r) -> 1 + max (altezza(l)) (altezza(r));;
84
85 let rec balanced albero=
86     match albero with
87     | Empty -> true
88     | Tr(x, l, r) -> balanced l && balanced r && abs(altezza(l) - altezza(r))<=1;;
89
90 (* radice-> sottoalbero sinistro -> sottoalbero destro *)
91 let rec preorder albero =
92     match albero with
93     | Empty -> []
94     | Tr(x, l, r) -> [x]@(preorder l)@(preorder r);;
95
96 (* sottoalbero sinistro-> sottoalbero destro -> radice*)
97 let rec postorder albero =
98     match albero with
99     | Empty -> []
100     | Tr(x, l, r) -> (postorder l)@(postorder r)@[x];;
101
102 (*sottoalbero sinistro -> radice -> sottoalbero destro*)
103 let rec inorder albero =
104     match albero with
105     | Empty -> []
106     | Tr(x, l, r) -> (inorder l)@[x]@(inorder r);;
107
108 exception CamminoNonEsistente
109 let path_coprente albero lista =
110     let rec aux visitati albero lista =
111         match albero with
112         | Empty -> if lista=[] then visitati else raise CamminoNonEsistente
113         | Tr(x, Empty, Empty) -> if lista=[] || lista=[x] then List.rev (x::visitati)
114         else raise CamminoNonEsistente
115         | Tr(x, l, r) -> try
116             aux (x::visitati) l (List.filter ((<>) x) lista)
117             with CamminoNonEsistente -> aux (x::visitati) r (List.filter
118                 ((<>) x) lista)

```

```

116                                     in aux [] albero lista;;
117 type col = Rosso | Giallo | Verde | Blu;;
118 type 'a col_assoc = (col * 'a list) list;;
119
120 exception NoColore
121 let rec colore x lista =
122   match lista with
123   [] -> raise NoColore
124   | (col, numeri)::rest -> if List.mem x numeri then col else colore x rest;;
125
126 exception Colori
127 let path_to x listaColori albero =
128   let rec aux visitati albero colorePrec=
129     match albero with
130     Empty -> raise Colori
131     | Tr(y, Empty, Empty) -> if x=y then List.rev (x::visitati) else raise Colori
132     | Tr(y, l, r) -> if not (visitati=[]) then
133       if colorePrec=(colore y listaColori) then raise Colori else
134         try
135           aux (y::visitati) l (colore y listaColori)
136         with Colori -> aux (y::visitati) r (colore y listaColori)
137       else
138         try
139           aux (y::visitati) l (colore y listaColori)
140         with Colori -> aux (y::visitati) r (colore y listaColori)
141     in aux [] albero Rosso;;
142
143 (* Alberi n-ari *)
144
145 type multi_expr = MultiInt of int
146                 | MultiVar of string
147                 | MultiDiff of multi_expr * multi_expr
148                 | MultiDiv of multi_expr * multi_expr
149                 | MultiSum of multi_expr list
150                 | MultiMult of multi_expr list
151
152 let e = MultiSum [MultiInt 5; MultiVar "x"]
153
154 let rec subexpr e e' = match e with
155   MultiInt _ | MultiVar _ | MultiSum [] | MultiMult [] -> e = e'
156   | MultiDiff(e1,e2) | MultiDiv(e1,e2) -> e=e' || subexpr e1 e' || subexpr e2 e'
157   | MultiSum(x::xs) | MultiMult(x::xs) -> e=e' || subexpr x e' || List.exists
158   (function x -> subexpr x e') xs
159
160 let rec subst e var e' = match e with
161   MultiInt _ -> e
162   | MultiVar x -> if x=var then e' else e
163   | MultiDiff(e1,e2) -> MultiDiff(subst e1 var e', subst e2 var e')
164   | MultiDiv(e1,e2) -> MultiDiv(subst e1 var e', subst e2 var e')
165   | MultiSum l -> MultiSum (List.map (function x -> subst x var e') l)
166   | MultiMult l -> MultiMult (List.map (function x -> subst x var e') l)
167 ;;
168
169 (* Visite alberi n-ari *)
170
171 type 'a ntree = Tr of 'a * 'a ntree list;;
172
173 let rec foglie_in_lista l = function
174   Tr(n,[]) -> List.mem n l
175   | Tr(_,ts) -> List.for_all (foglie_in_lista l) ts

```

```

175 (* radice -> sottoalberi *)
176 let rec preorder (Tr(x, figli)) = [x]@(List.flatten (List.map (preorder) figli));;
177
178 (* sottoalberi -> radice *)
179 let rec postorder (Tr(x, figli)) = (List.flatten (List.map (postorder) figli))@[x];;
180
181 (* sottoalbero sinistro -> radice -> sottoalbero destro *)
182 let rec inorder (Tr(x, figli)) =
183   match figli with
184   [] -> [x]
185   | y::rest -> (inorder y)@[x]@(List.flatten (List.map (inorder) rest));;
186
187 let rec tutte_foglie_costi albero =
188   let rec aux costo l albero =
189     match albero with
190     Tr(x, []) -> (x, costo+x)::l
191     | Tr(x, list) -> List.flatten(List.map (aux (costo+x) l) list)
192   in aux 0 [] albero;;
193
194 exception NoTree
195
196 let rec ramo_da_lista albero lista k =
197   let rec aux risultato albero lista =
198     match albero with
199     Tr(x, []) -> if x=k && lista=[x] then risultato@[x] else raise NoTree
200     | Tr(x, figli) -> let rec scorrifigli figli =
201       match figli with
202       [] -> raise NoTree
203       | z::rest ->
204         try
205           if (List.mem x lista) then aux (risultato@[x]) z
206         (List.filter ((<>) x) lista) else aux risultato z lista
207         with NoTree -> scorrifigli rest
208       in scorrifigli figli
209   in aux [] albero lista;;
210
211 exception NumeroNonEsistente
212 let rec colore x assoc_col =
213   match assoc_col with
214   [] -> raise NumeroNonEsistente
215   | (col, lista)::rest -> if (List.mem x lista) then col else colore x rest;;
216
217 exception NoRamo;;
218 let ramo_colorato x assoc_col (Tr(nodo,figli)) =
219   let rec aux visitati (Tr(nodo2, figli)) =
220     match figli with
221     [] -> if nodo2=x then (List.rev visitati) else raise NoRamo
222     | _ -> let rec scorrifigli figli =
223       match figli with
224       [] -> raise NoRamo
225       | (Tr(y,figli2))::rest ->
226         try
227           if (colore (List.hd visitati) assoc_col) = (colore y
228             assoc_col) then raise NoRamo
229           else aux (y::visitati) (Tr(y,figli2))
230         with NoRamo -> scorrifigli rest
231       in scorrifigli figli
232   in aux [nodo] (Tr(nodo, figli));;
233
234 (* Visite grafi *)

```

```
233 type 'a graph = ('a * 'a) list;;
234
235 (* Funzione successori *)
236 exception NonNelGrafo
237 let rec successori x grafo = if List.exists (function (y,z) -> x=y || z=x) grafo
238 then List.map (snd) (List.filter (function (y,_) -> x=y) grafo) else raise
239 NonNelGrafo;;
240
241 (* Funzione successori per grafi non orientati *)
242 let rec vicini x grafo = if List.exists (function (y,z) -> x=y || z=x) grafo then
243 List.map (function (y,z) -> if x=y then z else y)
244 (List.filter (function (y,z) -> x=y || x=z) grafo) else raise NonNelGrafo;;
245
246 (* Visita in profondità *)
247 let depth_first_search start grafo =
248   let rec aux visitati pendenti =
249     match pendenti with
250     [] -> List.rev visitati
251     | x::rest -> if List.mem x visitati then aux visitati rest
252                   else
253                     aux (x::visitati) (successori x grafo)@rest
254                   in aux [] [start];;
255
256 (* Visita in ampiezza *)
257 let breadth_first_search start grafo =
258   let rec aux visitati pendenti =
259     match pendenti with
260     [] -> List.rev visitati
261     | x::rest -> if List.mem x visitati then aux visitati rest
262                   else
263                     aux (x::visitati) rest@(successori x grafo)
264                   in aux [] [start];;
265
266 let test_connessi grafo nodo1 nodo2 =
267   let rec aux visitati pendenti =
268     match pendenti with
269     [] -> false
270     | x::rest -> if x=nodo2 then true
271                   else
272                     if List.mem x visitati then aux visitati rest
273                     else aux (x::visitati) ((successori x grafo)@rest)
274                   in aux [] [nodo1];;
275
276 exception CamminoInesistente
277 let rec cammino (listaNodi, listaArchi) lista nodo1 nodo2 =
278   if not (List.mem nodo1 listaNodi && List.mem nodo2 listaNodi) then raise
279   CamminoInesistente else
280   let rec aux visitati pendenti lista =
281     match pendenti with
282     [] -> raise CamminoInesistente
283     | x::rest -> if x=(List.nth lista ((List.length lista)-1)) && x=nodo2 then
284       List.rev (x::visitati)
285     else
286       if List.mem x visitati then aux visitati rest lista
287       else
288         if List.mem x lista then aux (x::visitati) (rest@(successori
289 x listaArchi)) lista
290         else aux visitati rest lista
```

```
287         in aux [] [nodo1] lista;;
288
289 exception NonEsiste
290 let nodo_predicato start grafo p =
291   let rec aux visitati pendenti =
292     match pendenti with
293     [] -> raise NonEsiste
294     | x::rest -> if p x then x
295     else
296       if List.mem x visitati then aux visitati rest
297       else
298         aux (x::visitati) (rest@(successori x grafo))
299   in aux [] [start];;
300
301 let is_primo x =
302   if abs(x)<2 then false else
303   let rec aux n=
304     if n*n > abs(x) then true
305     else if abs(x) mod n=0 then false
306     else aux (n+1)
307   in aux 2;;
308
309
310 exception NoCamminoDiPrimi
311 let rec cammino_di_primi grafo start goal =
312   let rec aux visitati pendenti =
313     match pendenti with
314     [] -> raise NoCamminoDiPrimi
315     | x::rest -> if x=goal && is_primo x then List.rev (x::visitati) else
316       try
317         if not(is_primo x) then raise NoCamminoDiPrimi
318       else
319         if List.mem x visitati then aux visitati rest
320         else aux (x::visitati) (rest@(successori x grafo))
321   with NoCamminoDiPrimi -> aux visitati rest
322   in aux [] [start];;
323
324
325 exception NoCiclo
326 let ciclo grafo nodo =
327   let rec aux visitati pendenti=
328     match pendenti with
329     [] -> raise NoCiclo
330     | x::rest -> if x=nodo then List.rev (x::visitati) else
331       if List.mem x visitati then aux visitati rest
332       else
333         try
334           aux (x::visitati) (successori x grafo)
335         with NoCiclo -> aux visitati rest
336   in aux [] [nodo];;
```